

Pearls AQI Predictor

A 100% serverless, end-to-end machine learning system
for forecasting air quality in Karachi

Project Report

A three-day (+24h / +48h / +72h) AQI forecasting platform built on free-tier infrastructure: Open-Meteo, Supabase, GitHub Actions, and Streamlit.

Live dashboard: <https://10pearls-karachi-aqi-predictor-v8ygbuwud8du7m3mun22cr.streamlit.app>

Source code: <https://github.com/Zee9710/10Pearls-Karachi-AQI-Predictor>

Location: Karachi (24.8607° N, 67.0011° E), Asia/Karachi

Forecast horizon: +24h, +48h, +72h (direct multi-horizon)

Champion model: Ridge regression at all three horizons

Abstract

This report documents the design and implementation of an end-to-end machine learning system that forecasts the Air Quality Index (AQI) for Karachi, Pakistan over a three-day horizon (+24h, +48h, +72h). The system is fully serverless and runs entirely on free tiers: raw weather and pollutant data are fetched from the Open-Meteo API, engineered into 29 features, and persisted in a Supabase feature store; five model families spanning statistical to deep-learning approaches are trained and benchmarked, with champions promoted to a git-based model registry; GitHub Actions automate the feature pipeline hourly and the training pipeline daily; and an interactive Streamlit dashboard loads features and models from the store to display live and forecasted AQI with SHAP-based explanations and hazardous-level alerts. The architecture follows the canonical MLOps pattern of a decoupled feature store and model registry, so that each stage is independently testable, replaceable, and schedulable. Ridge regression emerged as the champion at all three horizons, achieving an RMSE of **21.88** and an R^2 of **0.308** at +24h on a strictly chronological hold-out set, outperforming Random Forest, XGBoost, LightGBM, and an LSTM that proved data-starved at the current corpus size of roughly 8,900 hourly observations.

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Design Philosophy	3
1.3	Contributions	3
2	System Architecture	3
2.1	Serverless Stack	3
2.2	Data Contracts	4
3	Feature Pipeline	4
3.1	Step 1 — Fetch Raw Data	5
3.2	Step 2 — Compute the AQI Target	5
3.3	Step 3 — Engineer Features	5
3.4	Step 4 — Store	5
4	Backfill	6
5	Training Pipeline	6
5.1	Step 1 — Fetch from the Feature Store	7
5.2	Step 2 — Train and Evaluate	7
5.3	Step 3 — Register	7
6	Results	7
6.1	Champion per Horizon	8
6.2	Full Model Comparison	8
6.3	Accuracy Decay with Horizon	8
6.4	Discussion	9
7	CI/CD Automation	9
8	Web Application	9
8.1	Loads Model and Features from the Feature Store	10
8.2	Displays Predictions	10
8.3	Hazardous-AQI Alerts	10
9	Exploratory Data Analysis and Guidelines	10
10	Deliverables Checklist	10
11	Limitations and Future Work	11

1 Introduction

Air pollution is a persistent public-health concern in Karachi, one of the world’s most populous cities, where particulate matter routinely pushes the AQI into the “Unhealthy” band. Reliable short-term forecasts let residents and authorities plan exposure, schedule outdoor activity, and issue advisories before conditions deteriorate. The objective of this project is to predict the AQI for the next three days using a fully automated, serverless machine-learning pipeline that can run unattended and at zero cost.

1.1 Problem Statement

Given a continuously growing history of hourly weather and pollutant measurements for a fixed location, produce calibrated point forecasts of the AQI at lead times of 24, 48, and 72 hours, refresh those forecasts automatically as new data arrives, and surface them through an interactive dashboard with explanations and hazard alerts.

1.2 Design Philosophy

The solution adopts the canonical MLOps pattern built around a **feature store** and a **model registry**. Each stage is decoupled: the feature pipeline only *writes* features, the training pipeline only *reads* features and *writes* models, and the dashboard only *reads*. This separation of concerns yields three concrete benefits:

- **Independent testability.** Each component has a single responsibility and a well-defined data contract, so it can be run, validated, and debugged in isolation.
- **Independent scheduling.** The feature pipeline runs hourly to keep the store fresh, while the expensive training pipeline runs only daily.
- **Replaceability.** Any stage can be swapped (e.g. Supabase for another Postgres host, or the git registry for MLflow) without touching the others, because they communicate only through the store.

1.3 Contributions

This project delivers (1) a reproducible, idempotent feature-engineering pipeline that materializes 29 features into a SQL feature store; (2) a multi-model training and benchmarking harness spanning the statistical-to-deep spectrum with leakage-safe evaluation; (3) a git-based model registry with automated champion promotion; (4) fully serverless CI/CD orchestration; and (5) a production dashboard with SHAP explainability and hazardous-AQI alerting.

2 System Architecture

The platform is organized as four decoupled stages connected by two durable interfaces—the **feature store** and the **model registry**—with a serverless orchestration layer driving the schedule. Figure 1 shows the end-to-end data flow.

2.1 Serverless Stack

Every component runs on a free tier with no server to maintain.

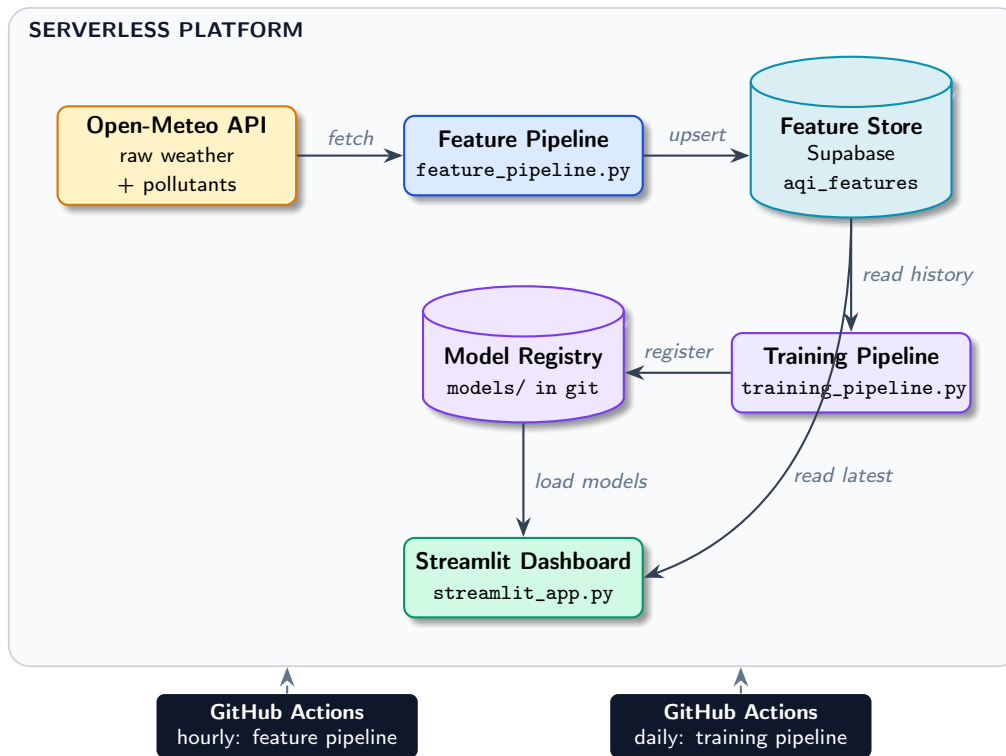


Figure 1: End-to-end architecture. Solid arrows are data flows; dashed arrows are scheduled triggers. The feature store and model registry are the only durable interfaces between stages.

Concern	Choice	Rationale
Raw data	Open-Meteo API	Free, no API key; provides air-quality, weather, and a historical archive end-point
Feature store	Supabase (Postgres + PostgREST)	Free tier, SQL-queryable, accessed over plain HTTPS
Model registry	<code>models/ in git</code>	Zero-cost, versioned, deploys with the app
Orchestration	GitHub Actions	Free cron scheduling, no server
Dashboard	Streamlit Community Cloud	Free hosting with a direct public URL

2.2 Data Contracts

The two interfaces are deliberately narrow. The feature store exposes a single table keyed on `datetime`; any producer that writes a conformant row is a valid feature pipeline. The model registry is a directory convention (`models/aqi_best_{h}h/`) containing a serialized estimator, its scaler, and a metrics file; any consumer that reads that convention is a valid serving layer. Because both contracts are storage-level rather than code-level, the stages share no runtime dependencies.

3 Feature Pipeline

The feature pipeline (`src/feature_pipeline.py`) performs four steps, shown in Figure 2.

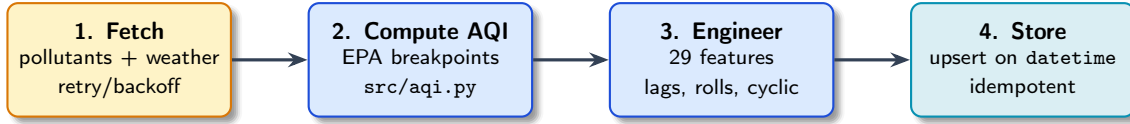


Figure 2: Four-step feature pipeline. Each run is idempotent: rows are upserted on the `datetime` key so re-runs never duplicate data.

3.1 Step 1 — Fetch Raw Data

Hourly pollutant data (PM2.5, PM10, ozone, NO₂, SO₂, CO) is pulled from Open-Meteo’s air-quality endpoint, and weather data (temperature, relative humidity, wind speed, precipitation, surface pressure) from the forecast/archive endpoints. A retry helper backs off exponentially on HTTP 429 and 5xx responses and honors the `Retry-After` header—important on the shared IP addresses used by cloud hosts, where rate limits are easy to trip. Requests that fail after the maximum number of retries raise, so a broken fetch surfaces as a failed CI run rather than a silently empty store.

3.2 Step 2 — Compute the AQI Target

The US-EPA AQI (0–500) is computed from raw pollutant concentrations in `src/aqi.py` using the standard piecewise-linear breakpoint formula. For a pollutant concentration C_p falling in the breakpoint interval $[BP_{Lo}, BP_{Hi}]$ with corresponding index endpoints $[I_{Lo}, I_{Hi}]$:

$$I_p = \frac{I_{Hi} - I_{Lo}}{BP_{Hi} - BP_{Lo}} (C_p - BP_{Lo}) + I_{Lo}, \quad \text{AQI} = \max_p I_p. \quad (1)$$

The overall AQI is the *maximum* sub-index across all pollutants, so the single worst pollutant determines the reported value. The index maps onto six health categories:

AQI Range	Category	Band
0–50	Good	■
51–100	Moderate	■
101–150	Unhealthy for Sensitive Groups	■
151–200	Unhealthy	■
201–300	Very Unhealthy	■
301–500	Hazardous	■

3.3 Step 3 — Engineer Features

Twenty-nine model inputs are derived from the raw signals and the computed AQI. They fall into six groups, summarized in Table 1.

3.4 Step 4 — Store

Features are upserted into the Supabase `aqi_features` table, keyed on `datetime`, so re-runs are idempotent and never create duplicate rows. The table schema is intentionally flat—one row per timestamp, one column per feature plus the AQI target—so it reads directly into a pandas DataFrame with no joins.

Group	Features	Why
Cyclical time	hour_sin/cos, dow_sin/cos, month_sin/cos, is_weekend	Sine/cosine encoding keeps 23:00 and 00:00 adjacent in feature space
Lags	aqi_lag_{1,24,48,72}h	Strong autocorrelation; yesterday's AQI is the best naive predictor
Rolling statistics	24h & 72h rolling mean, std, max (shifted)	Capture recent level and volatility without leaking the target
Change rate	aqi_change_24h	Day-over-day delta required by the assignment; encodes momentum
Interactions	temp_humidity, wind_pollution	Cross-effects (e.g. wind disperses pollutants)
Raw signals	6 pollutants + 5 weather variables	Direct meteorological and chemical drivers

Table 1: The 29 engineered features. All rolling and lag features are computed with shifts that guarantee no future information leaks into a row.

4 Backfill

To generate training data, the same feature script is run over a one-year range of past dates (`src/backfill.py`). This populated the feature store with approximately one year of hourly rows (around **8,900 records**). The backfill routes each date to the correct endpoint: the Open-Meteo *archive* endpoint for dates older than seven days, and the *forecast* endpoint for recent dates, since the two cover different time windows. Because the writer is idempotent, the backfill can be re-run safely to patch gaps without creating duplicates.

5 Training Pipeline

The training pipeline (`src/training_pipeline.py`) executes three steps, shown in Figure 3.

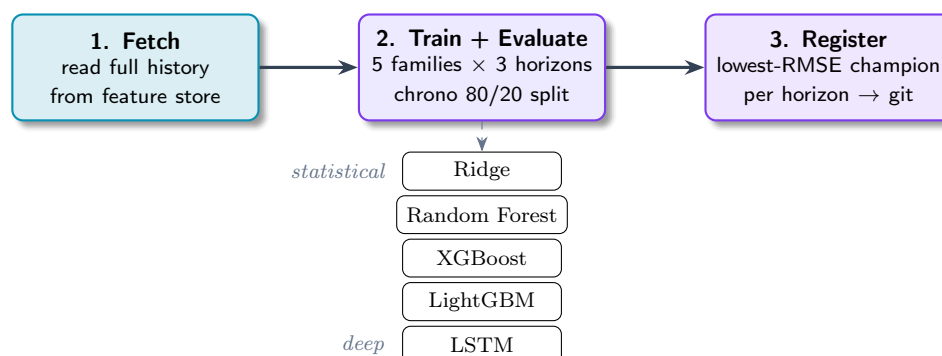


Figure 3: Three-step training pipeline. Five model families spanning the statistical-to-deep-learning spectrum are trained per horizon; the lowest-RMSE model becomes the champion.

5.1 Step 1 — Fetch from the Feature Store

The full feature history is read from Supabase. The training stage never touches the live raw API, satisfying the requirement that models train on *stored* features. This guarantees train/serve consistency: the dashboard and the trainer read identically engineered rows from the same source.

5.2 Step 2 — Train and Evaluate

For each horizon $h \in \{24, 48, 72\}$, the target is the AQI shifted forward by h hours, giving **direct multi-horizon** forecasting (a separate model per lead time) rather than recursive one-step rollouts, which would compound error. The data is split 80/20 *chronologically* (no shuffling) to respect time ordering and prevent look-ahead leakage—the held-out 20% is strictly the most recent tail, mimicking deployment. Five model families are trained:

Family	Type	Notes
Ridge Regression (RidgeCV)	Statistical / linear	L2-regularized; α chosen by built-in CV on standardized inputs
Random Forest	Bagged decision trees	Captures non-linear interactions, low variance
XGBoost	Gradient boosting	Strong tabular baseline
LightGBM	Gradient boosting	Histogram-based, fast
LSTM (TensorFlow/Keras)	Deep sequence model	Learns temporal dependence from windowed sequences

This spans the spectrum from statistical modelling to deep learning. Each model is scored on three metrics on the hold-out tail:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_i (\hat{y}_i - y_i)^2}, \quad \text{MAE} = \frac{1}{n} \sum_i |\hat{y}_i - y_i|, \quad R^2 = 1 - \frac{\sum_i (\hat{y}_i - \bar{y})^2}{\sum_i (y_i - \bar{y})^2}. \quad (2)$$

The lowest-RMSE model per horizon is promoted to champion.

5.3 Step 3 — Register

The champion model, its scaler, and its metrics are written to `models/aqi_best_{24,48,72}h/` and committed to git by the daily CI job, so the registry is versioned and deploys with the application.

6 Results

AQI is on the 0–500 EPA scale; all metrics are computed on the chronological 20% hold-out tail.

Champion

Ridge regression wins at every horizon. Best result: **RMSE 21.88**, MAE 15.53, R^2 0.308 at +24h.

6.1 Champion per Horizon

Horizon	Champion	RMSE	MAE	R^2
+24h	Ridge	21.88	15.53	0.308
+48h	Ridge	24.59	18.07	0.128
+72h	Ridge	25.47	18.99	0.065

6.2 Full Model Comparison

Table 2 reports RMSE for all five families across the three horizons; Figure 4 visualizes the same data.

Model	+24h	+48h	+72h
Ridge	21.88	24.59	25.47
Random Forest	22.95	28.86	28.05
XGBoost	23.74	28.30	27.10
LightGBM	24.24	28.64	27.49
LSTM	29.81	29.90	29.60

Table 2: RMSE by model and horizon (lower is better). Ridge is best in every column.

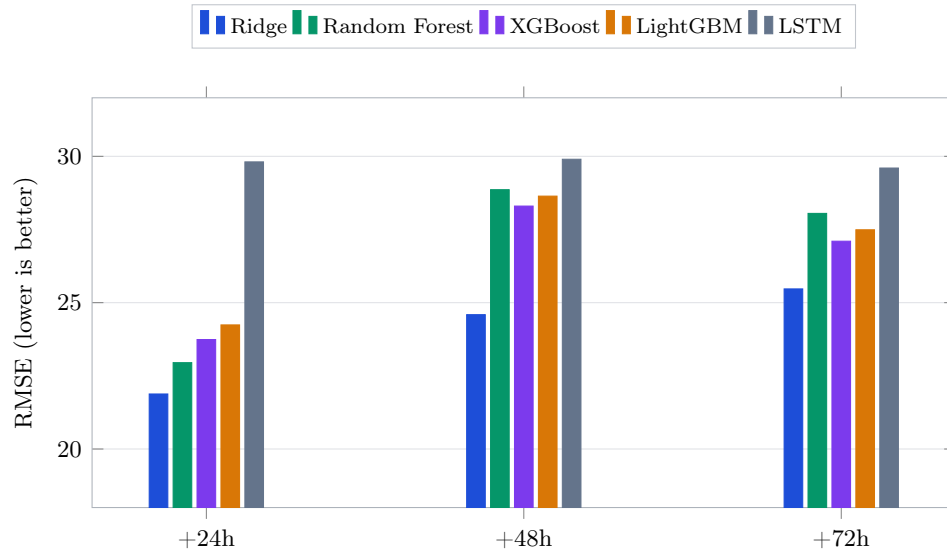


Figure 4: RMSE by model and horizon. Ridge (blue) is lowest everywhere; the gap widens at longer horizons where the ensembles and the LSTM degrade.

6.3 Accuracy Decay with Horizon

Predictive skill falls as the lead time grows, as expected. Figure 5 plots the champion's R^2 against horizon.

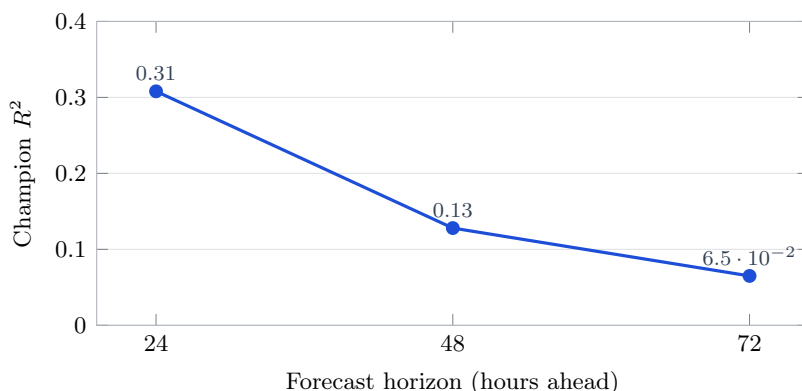


Figure 5: Champion R^2 drops from 0.31 at +24h toward the persistence baseline (≈ 0.07) at +72h.

6.4 Discussion

- **Ridge wins at every horizon.** With roughly 8,900 hourly rows and strongly auto-correlated lag features, the linear model generalizes better than the tree ensembles and the LSTM, which overfit (negative R^2 on the held-out tail at the longer horizons).
- **Accuracy decays with horizon**, as expected: +24h is the most predictable ($R^2 \approx 0.31$), while +72h approaches the persistence baseline ($R^2 \approx 0.07$).
- **The LSTM is data-starved.** Deep sequence models typically need far more data than is currently available; the LSTM is retained to demonstrate the full statistical-to-deep-learning spectrum and should be re-benchmarked as the feature store grows hourly.

7 CI/CD Automation

Two GitHub Actions workflows automate the system end to end, on independent schedules.

Workflow	Schedule	Action
hourly-feature.yml	0 * * * * (hourly)	Fetch the latest window, compute features, upsert to the store
daily-train.yml	0 3 * * * (daily)	Read the store, retrain all models, commit updated champions

Both authenticate to Supabase via repository secrets (`SUPABASE_URL`, `SUPABASE_KEY`) and require no server. The training job installs `requirements-train.txt` (adding TensorFlow on top of the base requirements) and pushes refreshed models back to the repository, so the next dashboard deploy automatically serves the newest champions. The hourly job keeps a lean dependency set so it starts quickly and stays well within free-tier minutes.

8 Web Application

The dashboard (`streamlit_app.py`) is deployed on Streamlit Community Cloud and styled after the IQAir interface.

8.1 Loads Model and Features from the Feature Store

On load, the app reads the most recent feature rows directly from Supabase and loads the champion models from the registry. If credentials are absent it falls back to a live Open-Meteo fetch so the demo still works; the active source is shown in the interface via a status pill.

8.2 Displays Predictions

The dashboard presents:

- Current AQI with a category ring, health advisory, and WHO PM2.5 multiplier.
- A live status pill showing the data source and last update time.
- Six pollutant concentration cards.
- Three-day forecast cards (+24h / +48h / +72h) with color-coded categories.
- A historical AQI trend chart (7 / 14 / 30 day windows).
- A model-performance section comparing RMSE across all five models per horizon, with the champion highlighted.
- A SHAP feature-importance section explaining which inputs drive each forecast.

8.3 Hazardous-AQI Alerts

Alert threshold

When a forecast crosses **AQI 150** (“Unhealthy”), the dashboard surfaces a prominent warning so users see the hazard before the affected period arrives.

9 Exploratory Data Analysis and Guidelines

The notebook `notebooks/eda.ipynb` examines pollutant distributions, feature correlations, and time-of-day and seasonal trends in the AQI signal, informing the choice of lag and rolling features. The table below maps each assignment guideline to its implementation.

Guideline	Implementation
EDA to identify trends	<code>notebooks/eda.ipynb</code>
Variety of models (statistical → deep learning)	Ridge, Random Forest, XGBoost, LightGBM, LSTM
SHAP / LIME explainability	SHAP in the dashboard; LIME in the optional Flask API
Alerts for hazardous AQI	Threshold alert at AQI 150

10 Deliverables Checklist

1. **End-to-end AQI prediction system** — complete (data → features → models → predictions).
2. **Scalable, automated pipeline** — complete (hourly and daily GitHub Actions; idempotent feature store).

3. **Interactive dashboard** — complete (live and forecasted AQI on Streamlit Cloud).
4. **Detailed report** — this document.

11 Limitations and Future Work

- **Forecast skill at long horizons.** $+72h$ R^2 is low; richer exogenous signals (wind direction, regional transport, traffic) could improve it.
- **LSTM needs more data.** As the hourly pipeline accumulates history, the deep model should be re-benchmarked; sequence models tend to overtake linear baselines once the corpus is large enough.
- **Single location.** The pipeline is parameterized by latitude and longitude in `src/config.py` and could be extended to multiple cities.
- **Git-based model registry.** Adequate at this scale; a managed registry such as MLflow would be the next step for model lineage in a larger team.
- **Point forecasts only.** The system reports a single value per horizon; prediction intervals (e.g. quantile regression) would better communicate uncertainty, especially at $+72h$.

Reproduce locally:

```
pip install -r requirements-train.txt
python -m src.backfill                # populate the feature store
python -m src.training_pipeline       # train + register champions
streamlit run streamlit_app.py        # launch the dashboard
```